

Ch. 2 - Authentication and Access Control

Secure software development and web security

R. Absil

Haute École Bruxelles-Brabant
École supérieure d'Informatique



September 17, 2026

Table of contents

- 1 Introduction
- 2 Authentication
 - Passwords
 - Dictionary attacks
 - Security tokens
 - Signature-based authentication
- 3 Access control
- 4 Implementing Authentication and Access control

Introduction

Not a simple tool

- We recalled the concepts of cryptographic hash functions, cipher and signature algorithms
- They have immediate uses in cryptography
- We also recalled some applications of them
- With these tools, the next step is authentication
 - Knowing “who is who”
- After authentication, access control must be enforced
 - Knowing “Who can do what under which conditions”

Authentication and Access control

- Many systems (computed related or not) have restricted access
- Some form of control is made on entry
- When a user wants to access a system, he authenticates
 - Credentials are presented
 - If the credential lies in some access control list, access is granted
 - If it does not, access is denied
 - Whatever happens, requests are logged to a database
- If access is forced, a detection system triggers an alarm

Two base principles

- 1 Record everything
- 2 Detect and deal with intrusions

- Once authenticated, access control is enforced through privileges

Authentication

Several types of authentication

- There are several types of authentication tools

Example

- Passwords
 - Security tokens
 - Cryptographic keys
-
- Make sure the identification is authentic
 - Establish a level of trust

Credentials

Question

- What shape can credentials take?
 - Often one of the following
 - Something you know
 - Something you have
 - Something you are
 - Multi-factor authentication: provide more than one piece of evidence
 - 2-FA: provide two, usually something you know and something you have
 - Zero-knowledge proof: prove that you know a secret or have a credential without disclosing it
 - Example: Server-Hello is signed in a TLS handshake

Authentication hardening

- Rely on existing and proven mechanisms
 - Don't reinvent the wheel
 - Error-prone
 - Standard protocols: JWT, OAuth, WebAuthn, OpenID
- Always use TLS
 - ... even in the context of end-to-end encryption
 - TLS gives *more* than confidentiality
- Force 2FA on registration and “credential lost”
 - Password and phone (not SIM-based)
 - Email isn't considered an “instance of something you have”
- Use CAPTCHA
 - Prevents brute-force attacks and slows down automated ones

Table of contents

1 Introduction

2 Authentication

■ Passwords

■ Dictionary attacks

■ Security tokens

■ Signature-based authentication

3 Access control

4 Implementing Authentication and Access control

Introduction

- Many systems are password protected
 - Mail servers, social networks, online accounts, etc.
- Instances of “something you know”

Drawback

- Low entropy: dictionary of common passwords
- Too hard to remember: “post-it on the screen”
- Password reuse

Advantage

- Simple

Transmission

- Obviously, a password cannot be transmitted as plain text
- An authentication request is often *not* signed
 - 1 A signature already proves identity
 - 2 Would require a second set of keys
 - And it would also cause “cultural problems”
- In practice, passwords are transmitted ciphered
 - Either with an asymmetric cipher,
 - Or with a session key (TLS)
- When received by the server, the password is unciphered and submitted to the system

Transmission and storage

- 1 Reception of a ciphered pair `login, password` through a TLS tunnel
- 2 The TLS termination deciphers the pair
- 3 Compute hash h of `password`
- 4 Check if the pair `login, h` is in the database

Improvement: salt

- Unique (random) per user, and public
- Usually, dedicated hash functions for passwords are used
 - Inherent use of salt, multi-passes, etc.
 - Protects against vicious dictionary attacks
 - Example: Argon2, Pbkdf2 with SHA156, etc.

Table of contents

1 Introduction

2 Authentication

- Passwords
- **Dictionary attacks**
- Security tokens
- Signature-based authentication

3 Access control

4 Implementing Authentication and Access control

Dictionary attack

- Very common attack against authentication
- Takes advantage that some passwords are very common

Principle

- 1 Build a list of common passwords
 - With their hashes out of several cryptographic hash functions
- 2 Look for a known hash in the table
- 3 If there is a match, the attack is successful

- Protection

- 1 Bound the number of request in time
- 2 Bound the number of failures
- 3 Make password policy more severe
- 4 Use salt

Password salting

Drawback

- Two identical passwords have the same hashes
 - Password reuse, same password for different users
- A single dictionary can crack every password

Solution: salt

- Affix a string to plain text passwords: the salt
 - Compute hash h' of this
 - Store the triple `login, salt,  $h'$`
-
- Different users with the same password now have different hashes
 - A dictionary attack needs a dictionary *per salt*

Striving for more

- We can still craft a dictionary for *a given salt*

Observation

- If the dictionary is *large enough*, likely success
 - Idea : try to invert the hash function on enough entries
- But it is impossible to simply brute-force the password space
- How to compromise?

Preprocessing compromise

- Assume we have a cryptographic hash function H with output of size n
- Let P be the set of all passwords accepted by the policy

Objective

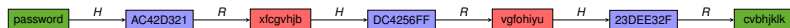
- Build up a data structure such that given an output h of H , it is possible to
 - either find $p \in P$ such that $H(p) = h$,
 - or state there is no such p in the data structure
- Brute force : compute $H(p)$ for all $p \in P$
 - Impossible in practice
 - Birthday attack

Precomputed hash chain

- Define a *reduction function* R mapping a hash to some $p \in P$
 - It is *not* required that R is the inverse of H (pre-image)
 - No constraint on collisions
 - R should uniformly distribute its output

Principle

- Alternate H and R to create "chains"



- We only store the head and the tail of a chain in the data structure

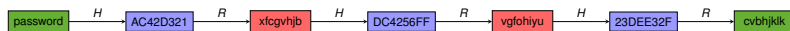
Password recovery

- Given some hash h , we apply R , then H , then R , then H , etc.
- Goal : at any point after computing R , discover the tail of some chain
- Build up the password from the head of the matching chain
 - 1 Starting from the head, we apply H , then R , then H , etc.
 - 2 If we get h , the previous element is the password we are looking for
 - 3 It is possible not to find h , since R can have collisions

Data structure

- Generate *a lot* of random passwords
- Compute *lengthy* chains
- For each chain, store the head and tail

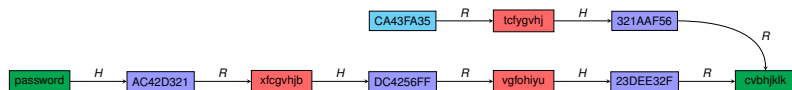
Illustration : success



- We are looking for the password matching the hash "DC4256FF"
- We apply R , and we get "vgfohiyu" : it is not a tail
- We apply H then R , we get "cvbhjklk" : it is a tail
- From "password" (the matching head), we apply H then R twice to get "DC4256FF"
- The password we were looking for is "xfcgvhjb"

Illustration : collision failure

- We are looking for the password with hash "CA43FA35"
- We apply R , we get "tcfygvhj"
- We apply H , we get "321AAF56"
- We apply R , we get "cvbhjklk"



- Building the password back from `password`, we *never* get "CA43FA35"
 - Failure
- Since R is far from being collision-free, chains can "merge"

Avoid collisions

Question

- How to avoid collisions in precomputed hash chains?

Solution

- Use several reduction functions
 - In practice, we use *a lot* of them
- Consequently, if two chains are to merge, they need to merge on *the same value on the same spot*.
 - Unlikely

Rainbow table

- Build k reduction functions R_i
 - In practice $\simeq 50\,000$
- From a (sufficiently large) set of random passwords, build a set of precomputed hash chains
 - Use H , then R_1 , then H , then R_2 , $\dots$, then H , then R_k
- Only store the head and tails of the chains (as pairs)
- Optimisations
 - Sort entries by tails
 - Delete duplicated tails

Advantage

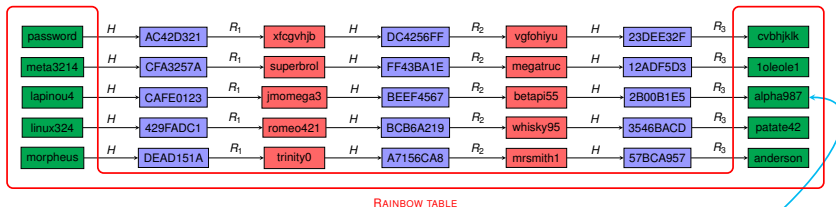
- 1 Little space used (compared to $|P|$), since we only store heads and tails
- 2 Low probability of collision on reduction functions

Rainbow attack

- Objective : crack some hash h
- How to
 - 1 Check whether $R_k(h)$ is the end of some precomputed hash chain
 - If it is, build up p back
 - 2 Check whether $R_k(H(R_{k-1}(h)))$ is the end of some precomputed hash chain
 - If it is, build up p back
 - 3 Iterate this technique until we check the complete chain
$$R_1 - H - R_2 - \dots - H - R_k$$
 - If it fails (again), the attack is a failure
- The bigger the table, the higher the chances of success
- Requires *a lot* of reduction functions to avoid collisions
- Requires a fitting implementation (BDD + SGBD, efficient hash tables, etc.)
- Salting prevents "mass cracking" of passwords
 - Along with usual advantages

Illustration

- 1 Build up the rainbow table
- 2 Check the last reduction
- 3 After i failures, check $R_{k-i} - H - R_{k-i+1} - \dots - H - R_k$
- 4 Build the password back if a match is found



Crack "BEEF4567"

1. Check R_k : "BEEF4567" $\xrightarrow{R_3}$ "rienavwr" $\rightarrow$ Failure
2. Check $R_{k-1} - H - R_k$: "BEEF4567" $\xrightarrow{R_2}$ "betapi55" $\xrightarrow{H}$ "2B00B1E5" $\xrightarrow{R_3}$ "alpha987" $\rightarrow$ Success

Reduction functions

- Objective : map a hash to a password
 - No no-collision requirement
 - Has to uniformly distribute the output

Construction example: build the i^{th} function

- Example 1
 - 1 Parse the hash as some integer x
 - 2 Return $(x + i) \bmod |P|$
 - 3 Map the output to a string, e.g., with a character table
- Example 2
 - 1 Pick the i^{th} combination of k characters from the hash
 - Pick k so the resulting string is properly bounded
 - 2 Compute a basis-change from the hash space to the password space

Table of contents

1 Introduction

2 Authentication

- Passwords
- Dictionary attacks
- **Security tokens**
- Signature-based authentication

3 Access control

4 Implementing Authentication and Access control

Overview

- A token does not authenticate its bearer by virtue of being associated with their identity
- If a user presents a valid token, the service treats its holder as authenticated

Primary constraint

- Security relies on *possession* of the token
 - The token must be difficult to guess or forge
-
- In practice, *unique* possession is hard to guarantee
 - We can categorise tokens into three sets
 - 1 Paper tokens: sets of (usually one-time use) codes written down
 - 2 Software tokens: relies on a software installed on the client computer/device
 - 3 Hardware tokens: requires physical device

Tokens in practice

- Often used to allow authentication to be delegated by another service
 - 1 Authenticate to your favourite IdP
 - 2 The IdP gives you a token
 - 3 You give the token to the service you want to use

Advantage

- The used service never sees your credentials
- Sometimes used to “sustain” authentication rather than “first time authentication”
 - Reduce the need to authenticate over and over

Asynchronous example: Lamport scheme

- Each generated password relies on
 - 1 some cryptographic hash function
 - 2 the previously generated password

Algorithm

- 1 Initially, the client
 - 1 chooses some (secret) s and a large (not secret) k
 - 2 computes the chain $p_0 = H^k(s) = H(H(H(\dots H(s) \dots)))$
 - 3 sends p_0 to the server (at login), who stores it
- 2 The first password required is $p_1 = H^{k-1}(s)$
 - p_1 is computed by the client from the start of the chain with s
 - if the client sends p , the server checks if $H(p) = p_0$
- 3 The second password required is $p_2 = H^{k-2}(s)$
 - ... even if authentication failed for p_1

Table of contents

1 Introduction

2 Authentication

- Passwords
- Dictionary attacks
- Security tokens
- Signature-based authentication

3 Access control

4 Implementing Authentication and Access control

Cryptographic key-based authentication

Scheme

- 1 The server challenges a nonce to a client
 - 2 The client signs the nonce and send it back
 - 3 The server checks the signature
-
- Prerequisite : the server must have the public key of the client
 - The keypair could be generated on registration
 - Usually performed on client-side
 - Private keys are stored ciphered locally, on client-side, for instance with a KDF

Certification-based authentication

Scheme

- The server is used as a private CA for its clients
- Each client has a certificate signed by the server
- The client
 - 1 presents his certificate to authenticate,
 - 2 signs the authentication request
- Usually, the client crafts a CSR on registration

KDF-based authentication

Scheme

- 1 The server challenges a nonce to the client
 - 2 The client use a KDF to derive a secret
 - 3 The client use the secret to generate a private key
 - Either directly (not compatible with every algorithm, such as RSA), or by seeding a generator
 - 4 The client signs the nonce
-
- The server must have the associate public key
 - Sent at registration

Advantages and drawbacks

- The three schemes are zero-knowledge proofs
- With cryptographic-key based and certification-based authentication, we can't log from a different device
 - Because the private keys and certificate are stored on a specific device
 - To log from multiple devices, we need additional steps
 - Usually, when we want to add a "new" device, we require the user to authenticate from a "master" device
- With KDF, we don't have to handle the storage of keys
 - Culture problems associated with passwords
 - We authenticate *users*, not *devices*
- A private key is usually password-encrypted and stored *locally* on a *trusted* computer
 - To access the private key, the user must provide the password
 - Decrypted private keys are often cached and released on demand

Access control

Overview

- Access control is often associated with authentication
- Only authenticated users are authorised inside the system
 - To grant access, the system must know who the user is
- Once users are authenticated, we need to enforce privileges
 - Not everybody can do everything
- Associates a user with a list of
 - actions he can perform
 - data he can access
- Accesses are mediated by a *guard* or *reference monitor*
 - Based on applicable policy, the guard makes a decision to grant or deny the requested access
- We must make sure no access can by-pass the guard

General principles

Base principle: least privilege

- Always give the least privileges needed to
 - authenticated users
 - services executed
- Objective: prevent innocent mistakes and privilege escalation

Separation of duties

- Separated roles and responsibilities are required to carry out actions
- Objective: prevent conflicts of interest

Broken access control: example

■ Message : login, pwd, usrId, resrcId

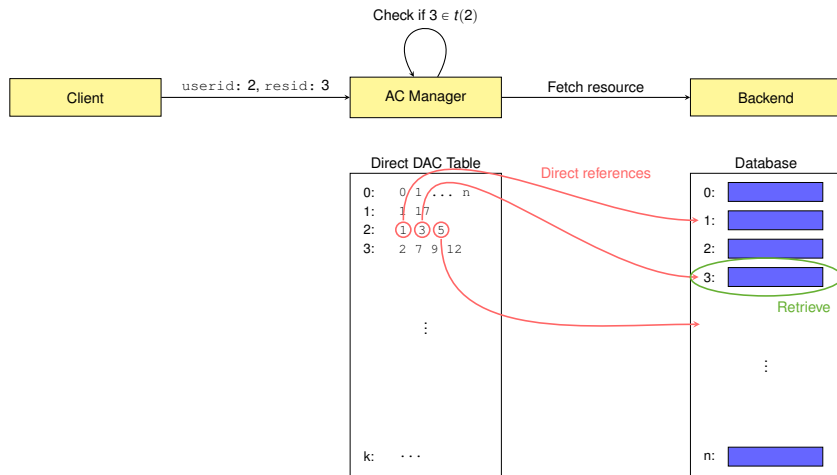
```
1 public Entry[] processRetrieval(SignedObject sm)
2     throws SecurityException, AuthenticationException, IllegalAccessException
3 {
4     if(sm.verify(s_publicKey, s_engine)) //check signature
5     {
6         SealedObject ciphered = (SealedObject)sm.getObject(); //recover ciphered object
7         Message plain = (Message)ciphered.getObject(c_privateKey); //remove cipher
8
9         boolean ok = auth_mgr.auth(plain.login(), hash.digest(plain.passwd().getBytes()));
10        if(ok) //user is authenticated successfully
11        {
12            boolean access = access_mgr.check(plain.getUserId(), plain.getResourceId());
13
14            if(access)
15                return db_mgr.retrieveEntries(plain.getUserId(), plain.getResourceId());
16            else
17                throw new IllegalAccessException("Resource_access_denied");
18        }
19        else
20            throw new AuthenticationException("Invalid_login/password");
21    }
22    else
23        throw new SecurityException("Invalid_message_signature");
24 }
```

A widespread vulnerability

Two problems

- Access a restricted resource or functionality by “guessing” a name
 - Peripheral security: no permission check after login
-
- For data, applications and API often use the real name or id of an object
 - For retrieval or page generation
 - For functions, URLs and function names are often easily predictable
 - Applications and API do not always check that a *registered* user has access to some resource
 - Manipulate parameters and attack (injection)
 - Careful code analysis often allows to detect and prevent such attacks

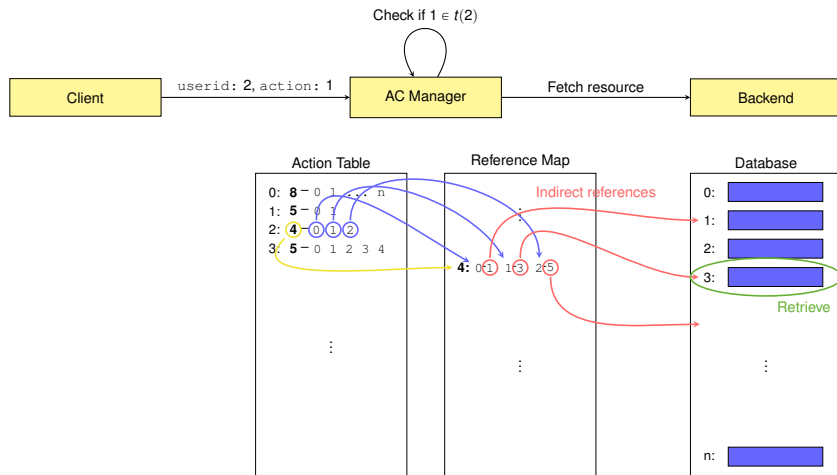
Direct references: illustration



Prevention

- Verify that *all* data and function references have appropriate defences
- After regular authentication,
 - for data reference, make sure the application (server) ensures the user is authorised for that data
 - for function reference, make sure the application ensures the user has required privileges for a call
- Each use of a reference from an untrusted source must include access control
- Use indirect references
 - Do not use DB keys
 - For user x , drop down a list of authorized resources / functions from 1 to k
 - This id denotes selection, *not a resource*
 - A user-depending table maps selection to resources / functions

Indirect references: illustration



How to?

- In practice, access control can be *very* complex
- Set of concepts, relationships, constraints that allow to express an access control policy in a formal way
- Formalised policies can be automatically processed
 - Validation: consistency, rule conflict or redundancy detection
 - Automatic enforcement
- Different models of access control
 - Discretionary access control
 - Mandatory access control
 - Role-based access control
 - Attribute-based access control
- Basic actions: read, write, execute, print/copy/download

Implementing Authentication and Access control

In practice

- Usually, authentication is performed by a (sometimes local) Idp
- Not by the backend-server
 - Separation of duties and delegation
 - Offers more options than standard password-based authentication
- The same is true for access control
 - A dedicated policy engine is used
- ... and for monitoring
- A reverse-proxy is used to
 - handle redirections
 - check tokens
 - act as a PEP
- Bridge to SSO and proper unsiloed-IAM

Illustration

